bot-finanzas

Personal finance bot: Discord slash commands, n8n and PostgreSQL

Repository: https://github.com/lyc4nthrope/bot-finanzas

Generated 2026-10-16

bot-finanzas

Personal finance bot: record income and expenses and ask for balances from Discord.

Stack

Piece	Tool
Chat channel	Discord slash commands
Automation	n8n (Docker; local first, then Hugging Face Space)
Database	PostgreSQL (local container, then Supabase)
Optional AI	OpenRouter free models

Setup

1. Copy the environment template and restrict its permissions:

   ```
   cp .env.example .env && chmod 600 .env
   ```

2. Fill in `.env`. Values marked `[SECRET]` stay out of chats and commits.

3. Start the local stack (Postgres, n8n and a temporary public tunnel) and apply migrations:

   ```
   docker compose --profile tunnel up -d
   for f in sql/0*.up.sql; do
     docker compose exec -T postgres sh -c 'psql -v ON_ERROR_STOP=1 -U "$POSTGRES_USER" -d
   "$POSTGRES_DB"' < "$f"
   done
   docker compose logs tunnel | grep -o 'https://.*trycloudflare.com'   # public URL
   ```

4. In n8n (http://localhost:5678) create the owner account and a Postgres credential named `Postgres local` (host `postgres`, database and user `finanzas`, password from `.env`).

5. Deploy the workflow and register the slash commands:

   ```
   scripts/deploy-workflow.sh workflows/discord-interactions.json discord-interactions
   scripts/register-commands.sh
   ```

6. In the Discord Developer Portal set **Interactions Endpoint URL** to `<tunnel URL>/webhook/discord-interactions`.

Production

```
Discord ──► Render (n8n, deploy/Dockerfile) ──► Supabase (PostgreSQL)
                ▲
     cron-job.org pings /healthz every 10 minutes so the free instance never sleeps
```

Piece	Where	Notes
n8n	Render free web service, Virginia	Built from `deploy/Dockerfile`; pushes do not auto-deploy
n8n state	Supabase, schema `n8n`	Role `n8n_app`; the Render instance is disposable
Money data	Supabase, schema `finance`	Role `bot_app`: no UPDATE, no DDL
TLS to Supabase	`deploy/supabase-ca.crt`	Certificates verified; never disable verification

Set up from scratch:

1. `scripts/setup-database.sh supabase --migrate` applies migrations and enables the roles.
2. `scripts/cloud-n8n.sh up`, then open http://localhost:5679 to create the owner account and a Postgres credential (`bot_app.<project-ref>`, SSL require) **before** the cloud instance exists, so its setup page is never publicly claimable.
3. `scripts/cloud-n8n.sh deploy workflows/discord-interactions.json <credential id> "<credential name>"`, then `scripts/cloud-n8n.sh down`.
4. Create the Render service (Docker, `deploy/Dockerfile`, health check `/healthz`) with the variables listed in ADR 0002, and point Discord's Interactions Endpoint URL to `<render URL>/webhook/discord-interactions`.

Decisions and measurements: `docs/adr/0002-render-instead-of-hugging-face.md`.

Tests

All tests run against the real local Postgres and n8n; nothing is mocked.

Script	What it proves
`scripts/test-schema.sh`	Table constraints reject invalid money data
`scripts/test-functions.sh`	Balances, periods (Bogota calendar), idempotency, undo
`scripts/test-roles.sh`	Least privilege: what `bot_app` and `n8n_app` can and cannot do
`scripts/test-bot.sh`	The workflow end to end: signatures, replies, database writes

Documentation

- `docs/contract.md`: what each command accepts and answers.
- `docs/workflow.md`: the n8n workflow node by node, with screenshots.
- `docs/adr/`: architecture decisions and their reasons.
- `docs/bot-finanzas.pdf`: all of the above in one PDF.

The Markdown files are the source of truth. After changing them or the workflow:

```
scripts/docs-screenshots.sh   # regenerates docs/images/ from a throwaway n8n
scripts/build-docs-pdf.sh     # regenerates docs/bot-finanzas.pdf
```

Layout

```
discord/    slash command definitions
docs/       contract and architecture decision records (ADR)
sql/        database migrations
workflows/  exported n8n workflows (JSON, no secrets)
scripts/    test and utility scripts
```

Bot contract (v1)

What the bot accepts and returns. Code must follow this file; change this file first.

Transport

- Discord sends each slash command as an HTTPS `POST` to `{WEBHOOK_URL}webhook/discord-interactions` (Discord "Interactions Endpoint URL").
- Every request carries `X-Signature-Ed25519` and `X-Signature-Timestamp`. The bot verifies the signature over `timestamp + raw body` (the exact bytes received, never re-serialized JSON) with the Discord public key.
  - Invalid or missing signature → `401`, nothing is processed.
  - Timestamp more than 300 seconds away from the server clock → `401` (replay protection).
- `type: 1` (PING) → `{"type": 1}`.
- `type: 2` (APPLICATION_COMMAND) → `{"type": 4, "data": {"content": "...", "flags": 64}}`.
  - `flags: 64` makes the reply visible only to the sender (financial data is private).
- Basic commands must reply in under 3 seconds (Discord limit).

Identity

- Each transaction belongs to the Discord user id that sent the command (`member.user.id` or `user.id`).
- A user only ever reads and changes their own transactions.

Amounts

- Option type `INTEGER`, whole Colombian pesos, `min_value: 1`, `max_value: 1000000000`.
- Why INTEGER and not NUMBER: in Colombia `20.000` means twenty thousand. A NUMBER option would silently read it as `20`; an INTEGER option makes Discord reject it before sending.
- Displayed as `$20.000` (es-CO format).

Commands

Slash command definitions live in `discord/commands.json`; their choices must match the database constraints in `sql/001_transactions.up.sql`.

`/gasto`

Option	Type	Required	Rules
`monto`	INTEGER	yes	1 to 1,000,000,000
`categoria`	STRING (choices)	yes	`mercado`, `comida`, `transporte`, `servicios`, `hogar`, `salud`, `deudas`, `otro`
`nota`	STRING	no	max 200 characters

Reply: `✅ Gasto registrado: $20.000 en mercado.\nGastado este mes: $350.000`

The same interaction delivered twice is stored once; the repeat gets `Ese movimiento ya estaba registrado.` (applies to `/ingreso` too).

`/ingreso`

Option	Type	Required	Rules
`monto`	INTEGER	yes	1 to 1,000,000,000
`categoria`	STRING (choices)	yes	`sueldo`, `ventas`, `regalo`, `otro`
`nota`	STRING	no	max 200 characters

Reply: `✅ Ingreso registrado: $1.500.000 (sueldo).\nSaldo total: $1.150.000`

`/saldo`

No options. Reply:

```
💰 Saldo total: $1.150.000
Este mes: ingresos $1.500.000 · gastos $350.000
```

`/resumen`

Option	Type	Required	Rules
`periodo`	STRING (choices)	no	`hoy`, `semana`, `mes` (default), `año`

Reply: totals of income and expenses for the period, plus expenses grouped by category, largest first. Periods use the `America/Bogota` calendar; `semana` starts on Monday.

`/deshacer`

No options. Deletes the sender's most recent transaction and shows what was deleted: `↩️ Eliminado: gasto de $20.000 (mercado).\nSaldo total: $1.130.000` If there is nothing to delete: `No hay movimientos para deshacer.` Negative balances display as `-$20.000`.

Errors

Case	Reply
Unknown command	`No conozco ese comando.`
Database unavailable	`⚠️ No pude guardar el movimiento. Intenta de nuevo en un momento.`
Any other failure	`⚠️ Algo salió mal. Intenta de nuevo.`

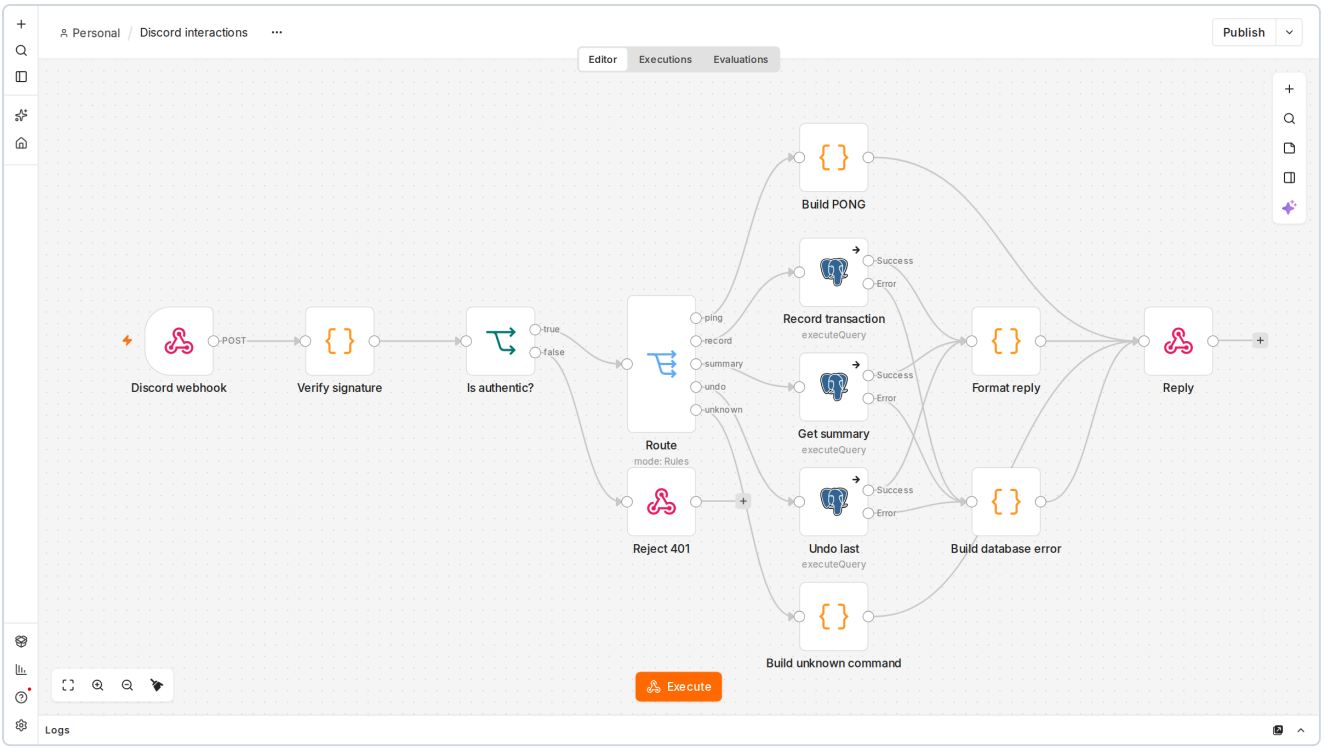
The bot never confirms a transaction that was not stored.

Out of scope for v1

- Free-text registration with AI (`/registrar` , phase 6).
- Editing past transactions (use `/deshacer` and register again).
- Budgets, alerts, voice notes.

The n8n workflow, node by node

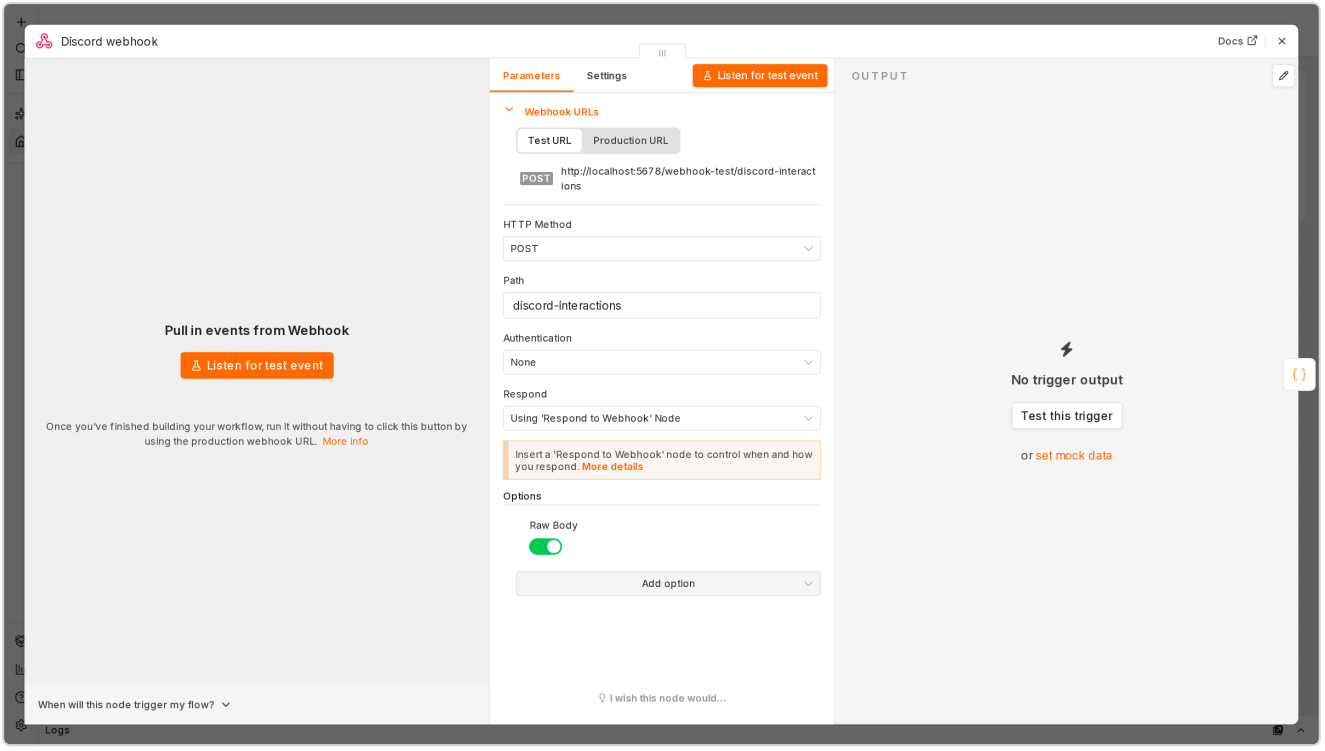
File: `workflows/discord-interactions.json` (13 nodes). Screenshots come from a throwaway n8n instance that imported this exact file; the Postgres credential shown is a placeholder.



The flow: Discord sends every slash command to the webhook. The bot checks the request really comes from Discord, routes it by command, calls one SQL function, and answers in the same HTTP response. Business rules live in PostgreSQL (`sql/`); n8n only adapts Discord to the database.

1. Discord webhook

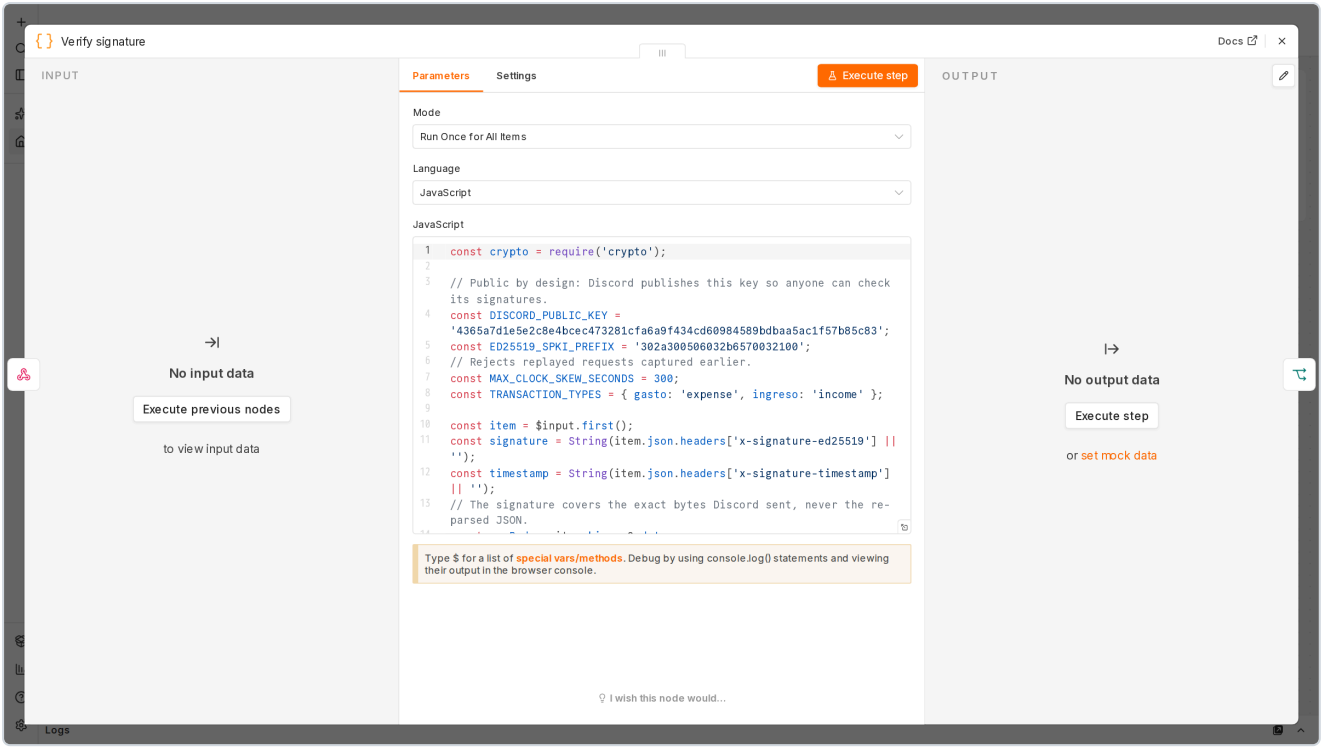
Receives Discord's `POST` on `/webhook/discord-interactions`. The **Raw Body** option keeps the exact bytes Discord sent, which the signature check needs. Responses are produced later by a *Respond to Webhook* node, so the HTTP request stays open until the bot has an answer.



2. Verify signature

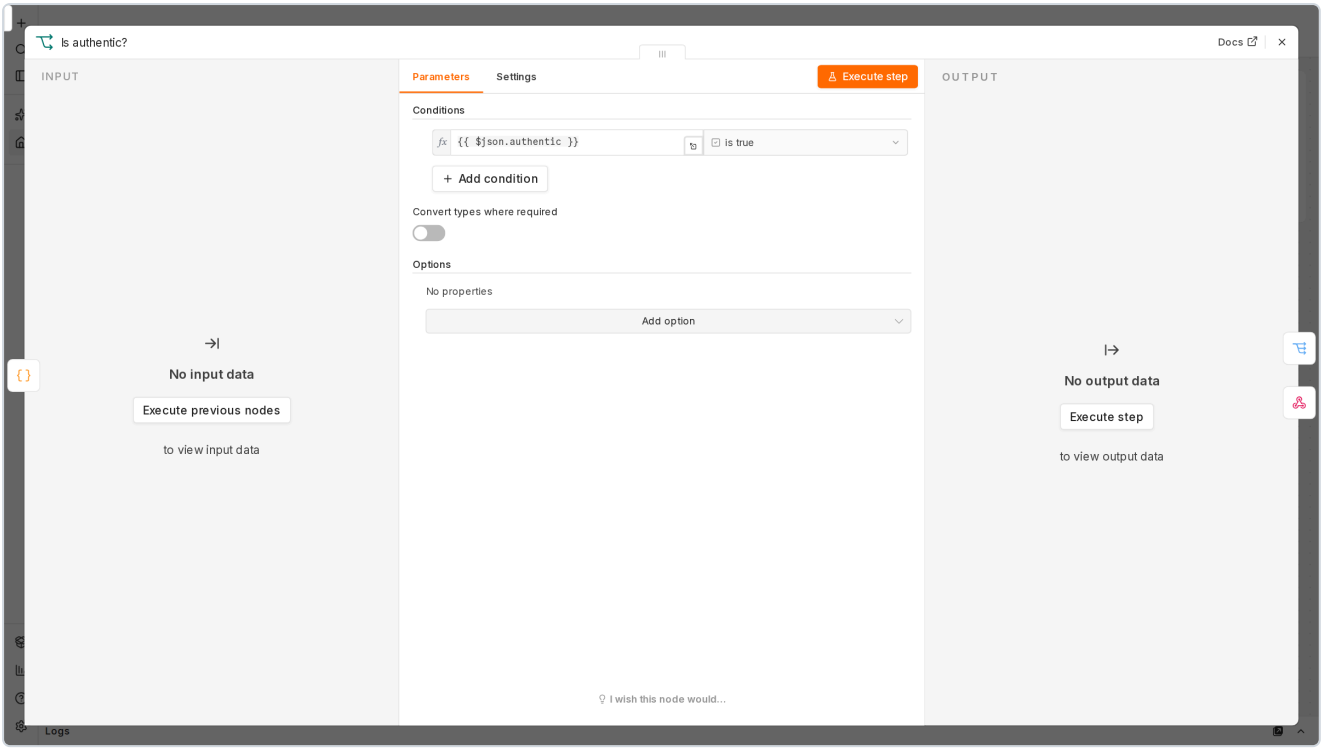
JavaScript (Code node). Discord signs `timestamp + raw body` with Ed25519; this node verifies it with Discord's **public** key, rejects timestamps older than 5 minutes (replayed requests), and extracts the command, its options and the Discord user id.

Why the raw bytes: n8n's parsed JSON changes values such as `1.50` to `1.5`, so a check against re-serialized JSON would reject genuine requests.



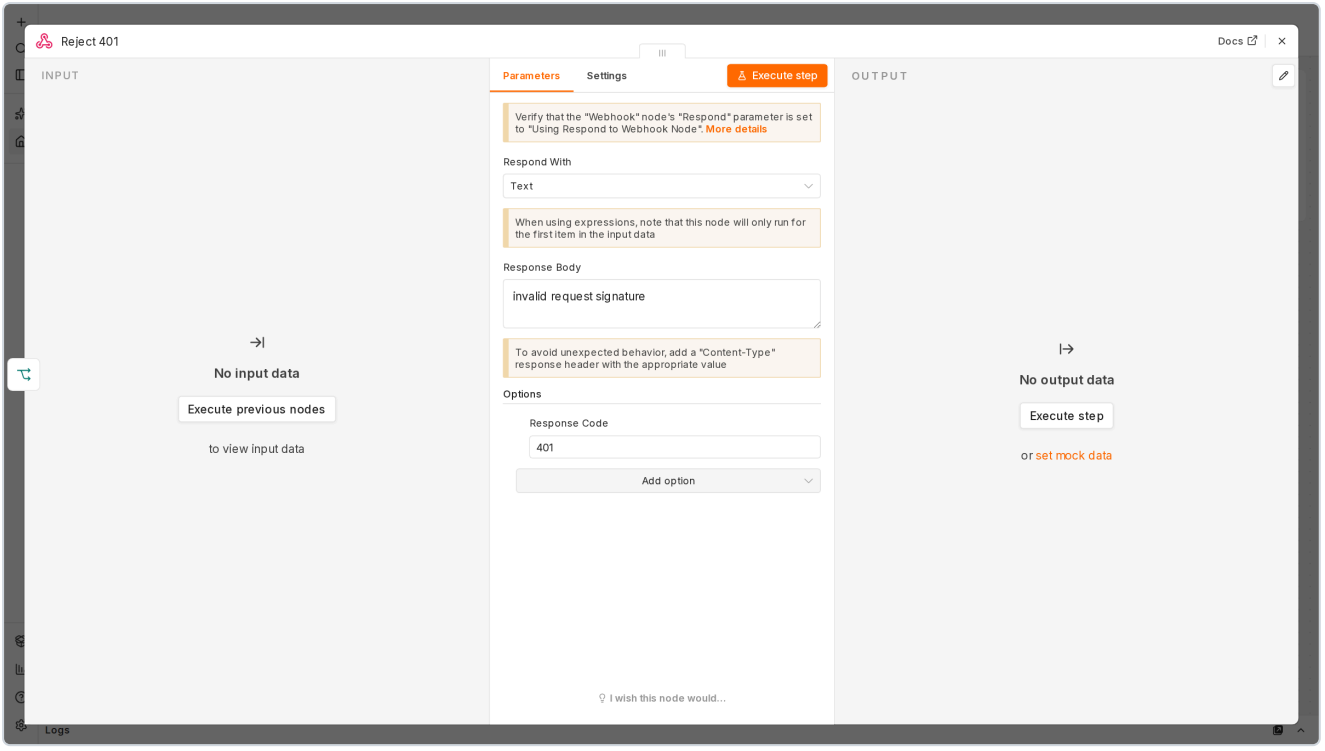
3. Is authentic?

`If` node on the `authentic` flag. Anything not signed by Discord goes to *Reject 401* and never reaches the database.



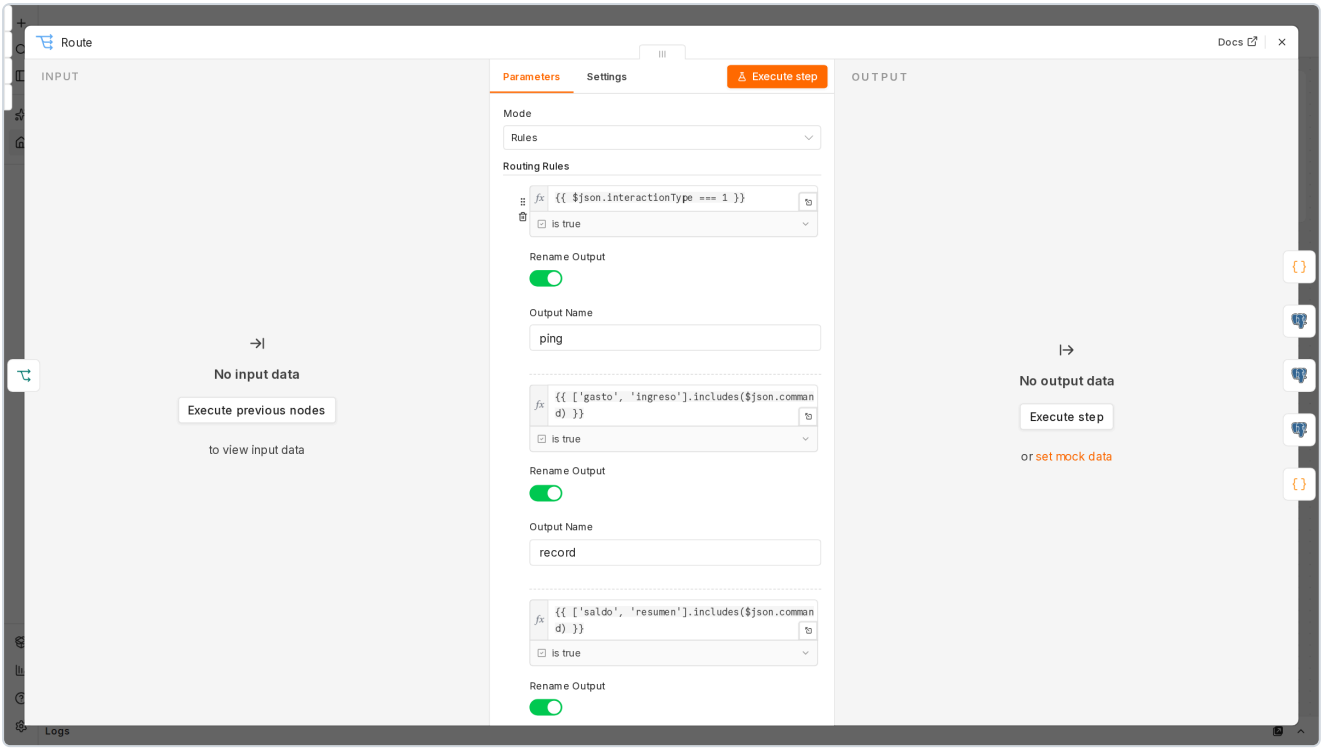
4. Reject 401

Answers `401 invalid request signature`. Discord itself checks this behaviour when the Interactions Endpoint URL is saved: it sends a request with a bad signature and refuses the URL unless the answer is 401.



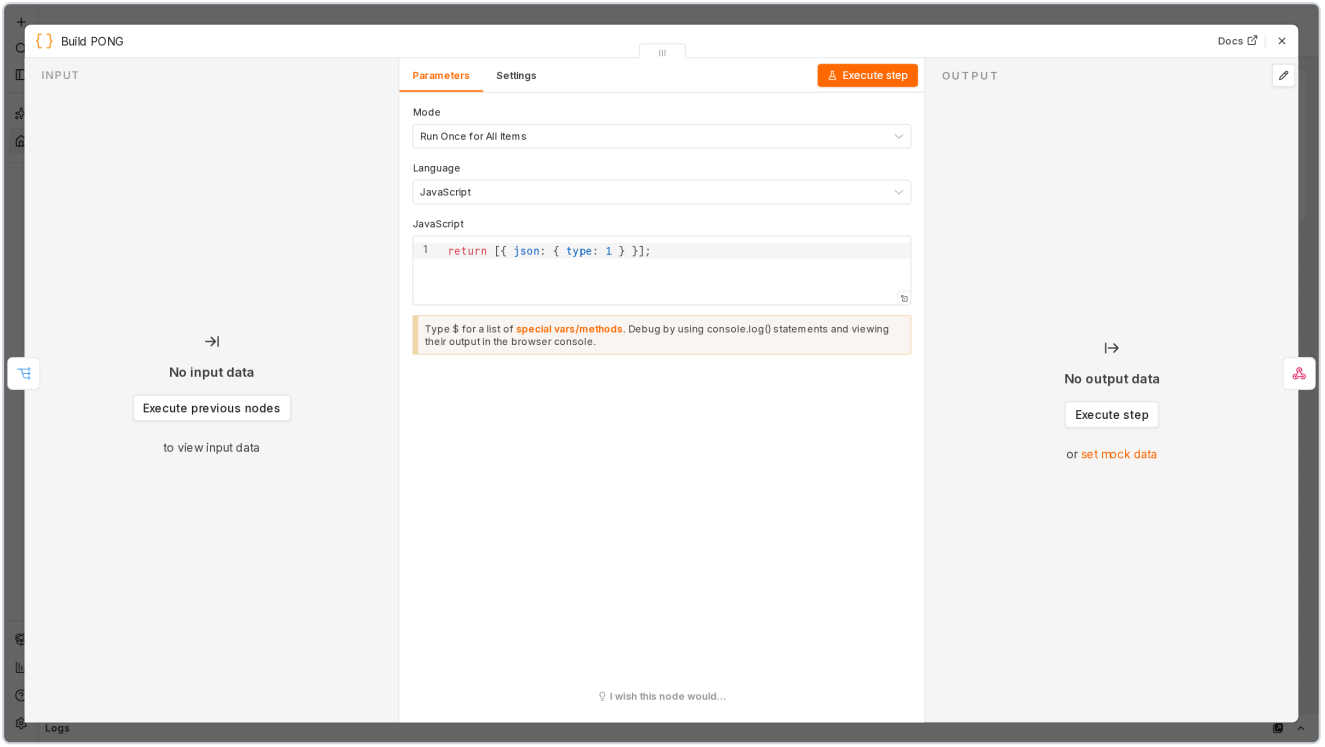
5. Route

`Switch` node with five outputs: `ping` (Discord's health check), `record` (`/gasto`, `/ingreso`), `summary` (`/saldo`, `/resumen`), `undo` (`/deshacer`) and `unknown` (fallback).



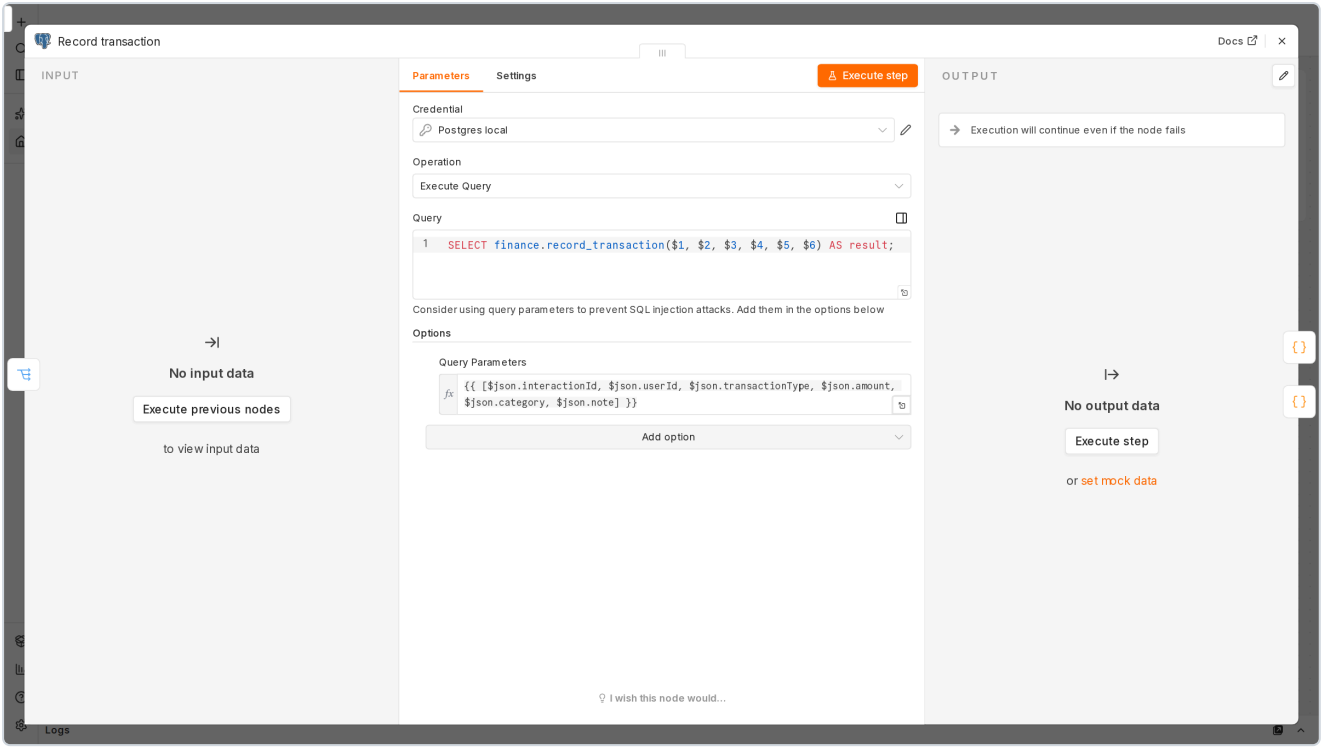
6. Build PONG

Answers Discord's `PING` with `{"type": 1}`.



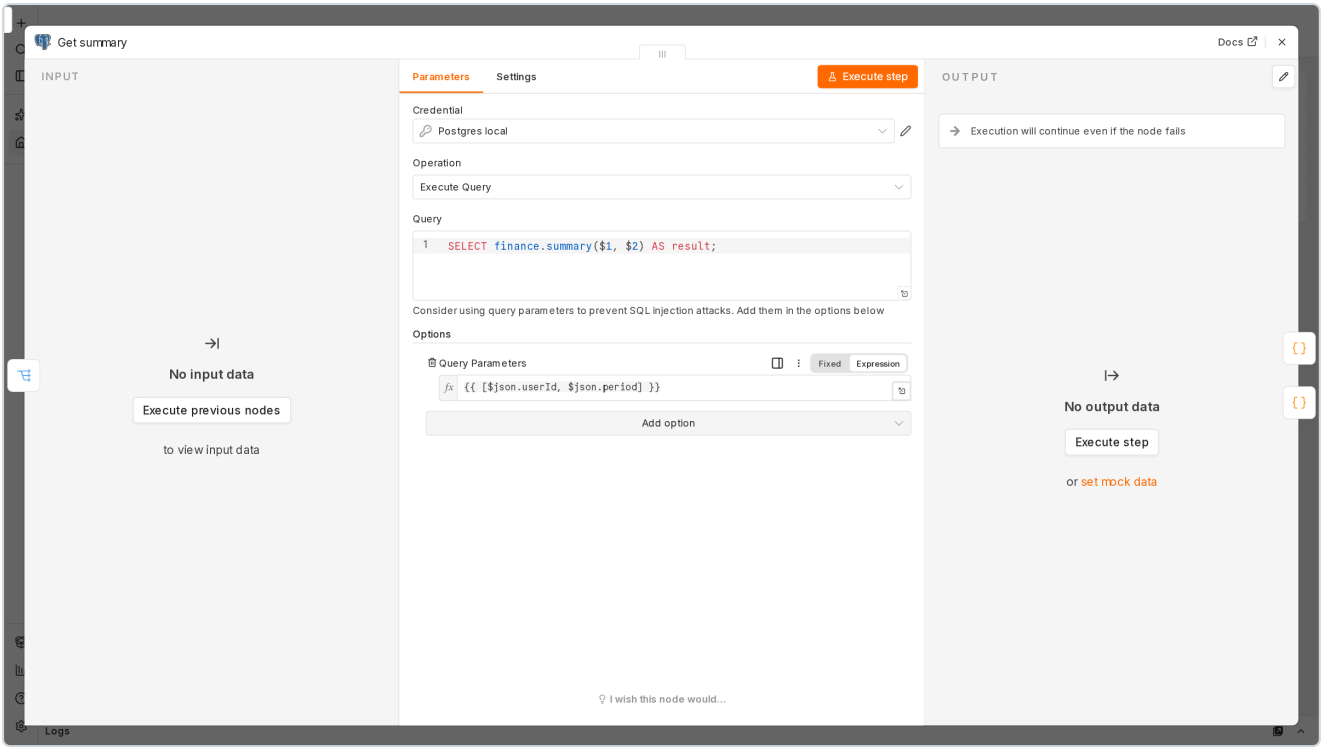
7. Record transaction

Calls `finance.record_transaction($1, ... $6)`. Values travel as **query parameters**, never pasted into the SQL text, which rules out SQL injection; a note such as `arepa, tinto` is stored with its comma intact. The node's error output leads to *Build database error*, so a failed insert is never confirmed to the user.



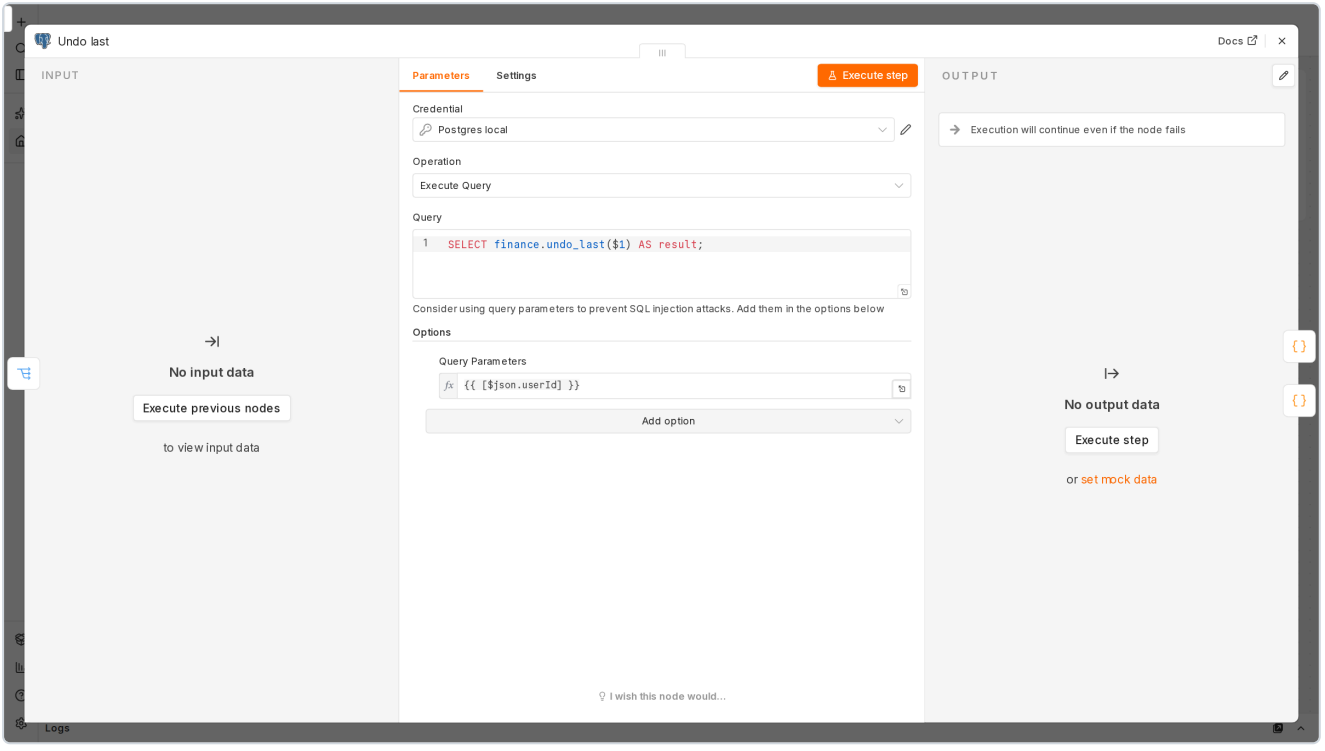
8. Get summary

Calls `finance.summary(user, period)` for `/saldo` (current month) and `/resumen` (`hoy`, `semana`, `mes`, `año`, on the Bogota calendar).



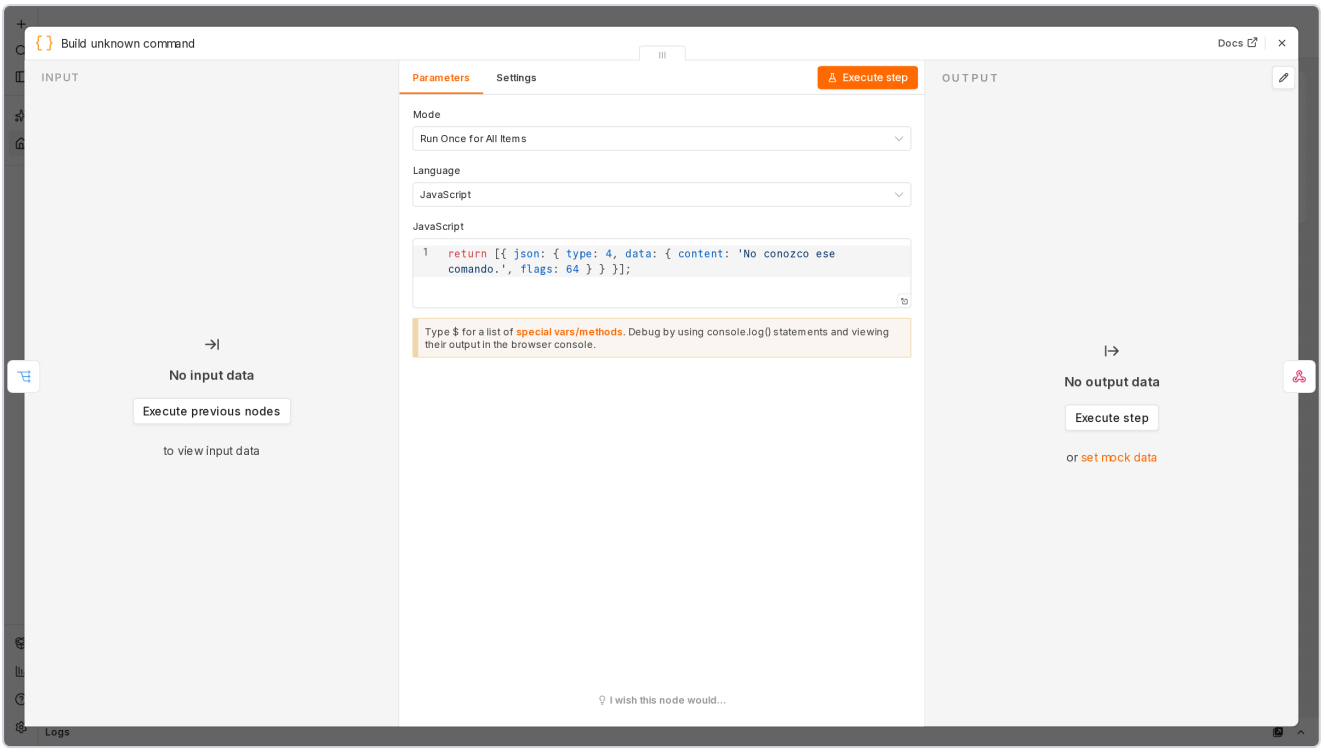
9. Undo last

Calls `finance.undo_last(user)`, which deletes only the sender's most recent transaction.



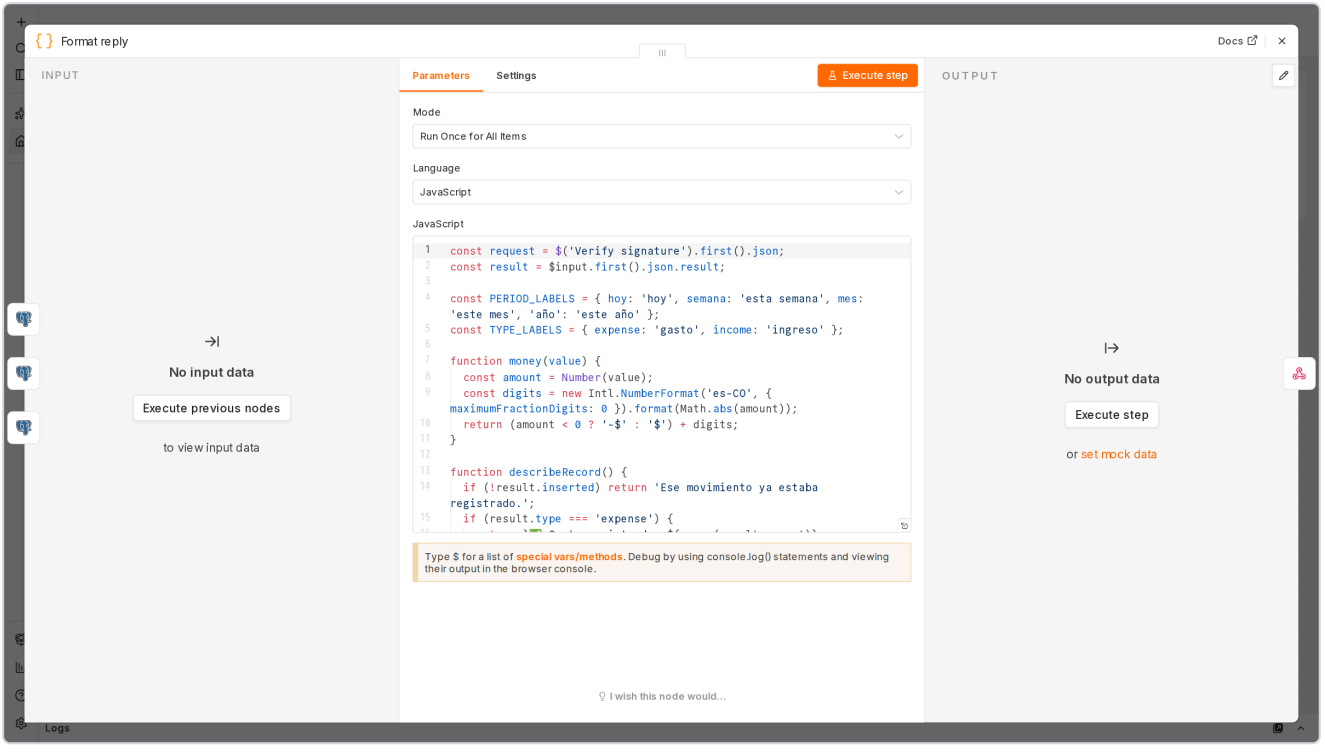
10. Build unknown command

Answers `No conozco ese comando.` for commands the bot does not know.



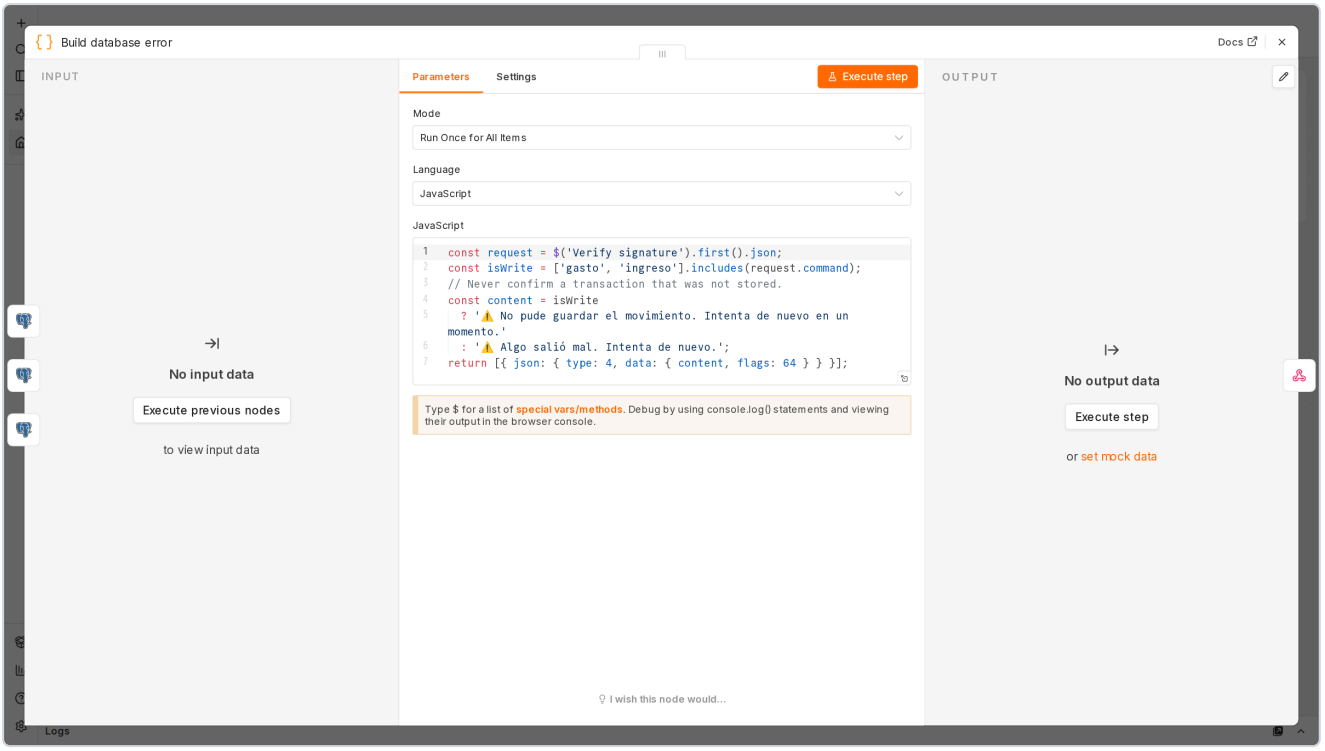
11. Format reply

Turns the SQL result into the Spanish messages defined in `docs/contract.md`, formats money as Colombian pesos (`$1.500.000`, negatives as `-$20.000`) and sets `flags: 64` so only the sender sees the reply.



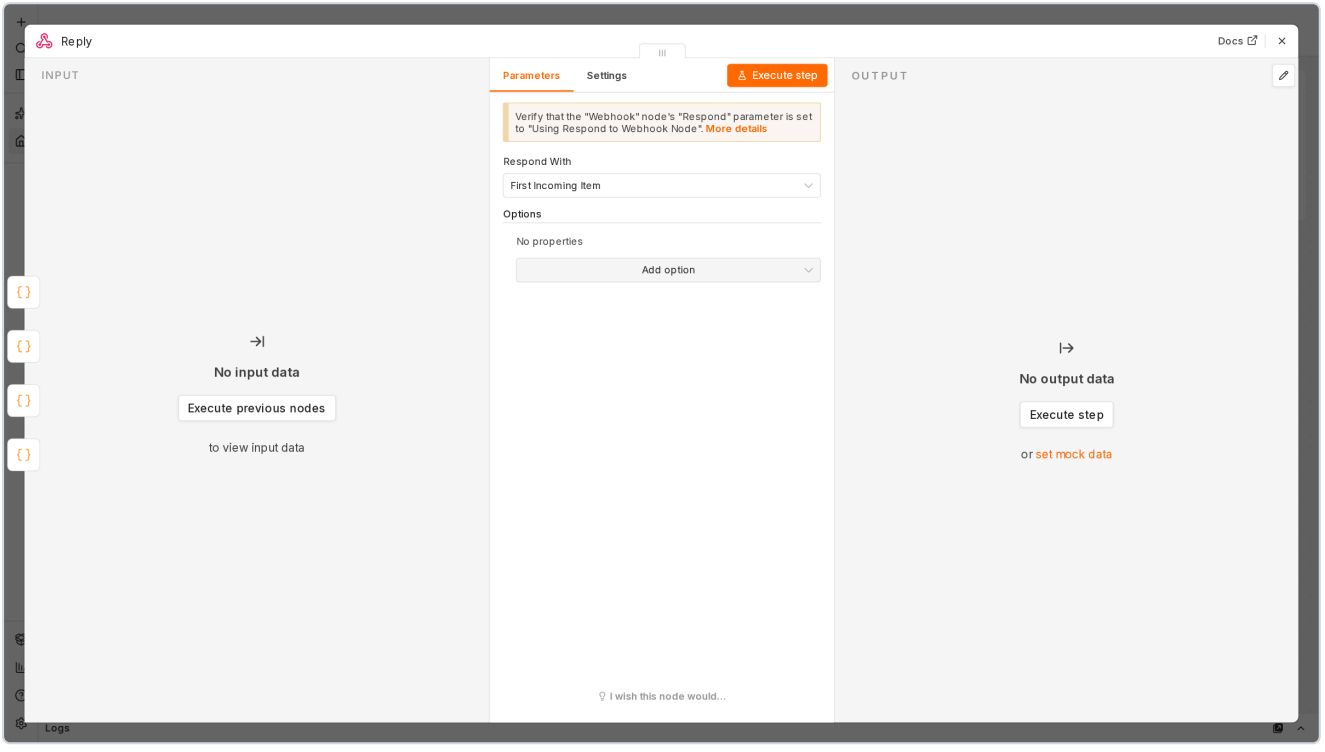
12. Build database error

Runs when a Postgres node fails. For `/gasto` and `/ingreso` it answers `⚠️ No pude guardar el movimiento...`; otherwise `⚠️ Algo salió mal...`.



13. Reply

Respond to Webhook node that sends whichever message reached it back to Discord. Discord drops answers that take more than 3 seconds; the end-to-end tests enforce a 2.5 second budget.



ADR 0001: Discord + n8n + PostgreSQL stack

- Status: accepted
- Date: 2026-10-16

Context

A finance bot for personal use must record income and expenses from a chat app and answer balance questions. Hard constraint: zero cost.

Decision

- **Channel:** Discord slash commands, received through the Interactions Endpoint URL.
- **Automation:** n8n in Docker. Develop locally, deploy to a Hugging Face Docker Space.
- **Storage:** PostgreSQL. Local container for development, Supabase for deployment. n8n's own state also lives in Supabase because the Space disk is ephemeral.
- **Keep-alive:** an external scheduler (cron-job.org) pings the Space; free Spaces sleep after 48 hours without traffic.
- **AI (optional, later):** OpenRouter free models, only for free-text input.

Alternatives considered

Option	Why rejected
Telegram	Hugging Face Spaces reportedly cannot resolve `api.telegram.org`; sign-up may require paying for SMS verification in some countries
WhatsApp Cloud API	Requires a Meta Business setup; heavy for a first version
WhatsApp via Evolution API	Unofficial; risk of the phone number being banned
Obsidian	Local Markdown files, no API a bot can use reliably
Notion	Works, but aggregating amounts is awkward compared with SQL
Free-text messages on Discord	n8n has no official trigger for them; needs a persistent gateway connection

Consequences

- Positive: free, official APIs, signed requests, structured input validated by Discord.
- Negative: users need a Discord account; replies must arrive within 3 seconds.
- Risk: outbound calls from Hugging Face to `discord.com` are unverified. A connectivity probe runs before deployment; basic commands avoid outbound calls entirely.

ADR 0002: Host n8n on Render instead of Hugging Face

- Status: accepted (supersedes the hosting part of ADR 0001)
- Date: 2026-10-16

Context

ADR 0001 planned to run n8n in a free Hugging Face Docker Space. When creating the Space, Hugging Face answered: "hosting Gradio and Docker Spaces on free cpu-basic requires a PRO subscription". Since July 2026 only Spaces created earlier keep the free tier.

Decision

Run n8n as a free Render web service built from `deploy/Dockerfile`, in the Virginia region (same AWS region as the Supabase project). An external scheduler pings it so the free instance does not sleep.

Alternatives considered

Option	Why not chosen
Hugging Face PRO (~9 USD/month)	The project must cost nothing
Oracle Cloud Always Free	Requires a card; ARM capacity is often unavailable; more operations work
Koyeb	Free tier closed to new users after its acquisition (Feb 2026)
Developer PC with a tunnel	The bot would stop whenever the PC is off

Measurements (Render free limits emulated locally: 512 MB, 0.1 CPU)

- Memory: 347 MB idle, no out-of-memory kills.
- Startup: about 110 s until `/healthz` answers.
- Reply time: 2.0 to 4.9 s, half above Discord's 3 s limit.

Measurements on the real Render service (2026-10-16)

- Webhook path with signature check, warm, 20 requests from Colombia: median 0.70 s, p90 0.98 s, max 1.28 s (about 0.3 s of that is network from Colombia).
- First request after a restart: about 22 s (the code-node runner loads on first use).
- `docker --cpus 0.1` turned out much stricter than Render's real CPU, so the local emulation overestimates reply times.
- The code-node runner needs ~47 s to start, more than n8n's default 30 s grant token: `N8N_RUNNERS_GRANT_TOKEN_TTL=180` is set in `deploy/Dockerfile`.

Consequences

- Synchronous replies fit Discord's 3 s limit for now. If they stop fitting, switch to Discord's deferred responses (acknowledge at once, edit the message later). The e2e suite enforces a 2.5 s reply budget so regressions are detected before users see them.
- A restart or deploy makes the bot unavailable for about two minutes.
- The keep-alive ping relies on Render tolerating it; if not, the bot answers late after 15 minutes without traffic.
- Production n8n state lives in Supabase (schema `n8n`), so the Render instance is disposable: redeploying it loses nothing.